Bloque 1: Concurrencia y Ciclo de Vida (Threads)

1. **Monitor de procesos en planta industrial:** Implemente un sistema en Java que lance 10 threads para simular sensores de temperatura en una matriz de producción, donde cada hilo genere valores aleatorios y finalice su ejecución únicamente si detecta un sobrecalentamiento superior a 100°C, informando en consola qué sensor se detuvo mientras el resto de la planta sigue operando con normalidad.
2. **Simulador de carrera en la Ruta 64:** Diseñe un programa con 3 threads que representen vehículos viajando de **Quillota a Viña del Mar** con tiempos de llegada variables y utilice el método `join()` en el hilo principal para bloquear el mensaje "¡Todos llegaron a la meta!" hasta que el último vehículo haya terminado el recorrido, sin utilizar `Thread.sleep()` en el proceso principal.
3. **Diferencia entre `start()` y `run()` en el servidor:** Cree un programa que intente ejecutar tres tareas de procesamiento de datos llamando directamente al método `.run()` para observar cómo se ejecutan secuencialmente en el hilo `main`, y luego modifique el código utilizando `.start()` para demostrar mediante `Thread.currentThread().getName()` cómo Java asigna hilos independientes para trabajar en paralelo.

Bloque 2: Sincronización y Exclusión Mutua

4. **Cuenta compartida para gastos comunes:** Desarrolle un sistema donde dos habitantes de un departamento intenten realizar retiros simultáneos de una misma cuenta bancaria para pagar el arriendo, programando primero una versión sin protección para evidenciar cómo el saldo puede quedar en valores negativos inconsistentes y luego utilizando `synchronized` para asegurar que el banco procese una transacción a la vez.
5. **Contador de votos para centro de alumnos:** Programe un escenario donde 100 threads incrementen una variable compartida que representa los votos de una urna electrónica, comparando un primer resultado sin control donde se pierden votos por condiciones de carrera, con una versión protegida que garantice que el conteo final sea exactamente 100 tras la ejecución de todos los hilos.
6. **Prevención de Deadlock en bodega:** Implemente un programa que fuerce un bloqueo mutuo (deadlock) mediante dos operarios de bodega (threads) que intenten adquirir una "Grúa" y un "Escáner" en orden inverso, y luego reestructure el código para que ambos soliciten los recursos siguiendo una jerarquía estricta que elimine la posibilidad de que el sistema se congelé.

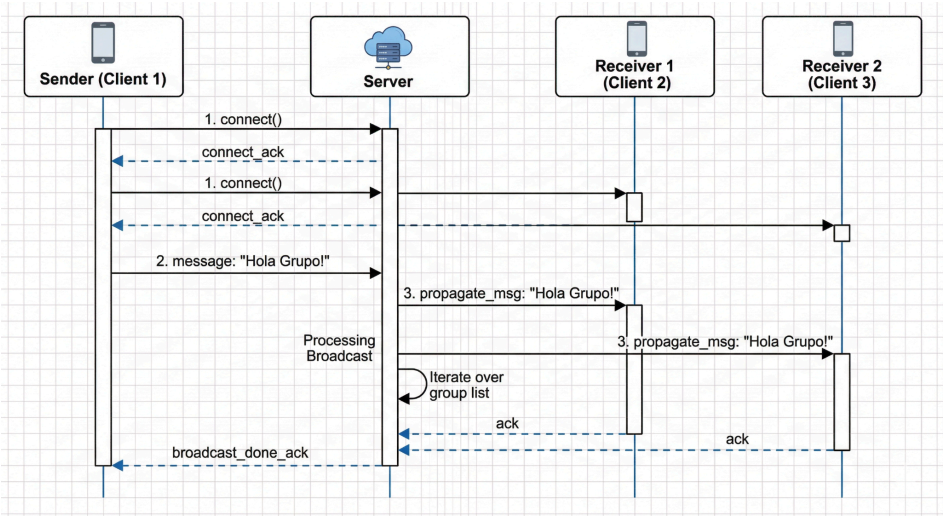
Bloque 3: Comunicación con Sockets

7. **Servidor de eco multihilo Santiago-Quillota:** Construya un servidor basado en `ServerSocket` que acepte conexiones simultáneas de clientes ubicados en distintas ciudades, delegando cada conexión a un hilo dedicado que reciba una cadena de texto, la invierta (como un eco procesado) y la devuelva al emisor sin que la demora de un cliente afecte la respuesta de los demás.
8. **Protocolo de login para sistema municipal:** Desarrolle una arquitectura cliente-servidor donde el usuario envíe sus credenciales en formato `usuario:password` y el servidor responda "OK" o "ERROR", integrando bloques `try-catch` para asegurar que el servidor no se caiga si un ciudadano pierde la conexión a internet repentinamente durante el proceso.
9. **Transferencia de manuales por socket:** Implemente un servidor que abra un archivo de texto de un "Manual de Procedimientos" y transmita su contenido línea por línea a un cliente que lo solicita desde una sucursal remota, asegurando que el cliente reciba el flujo completo y lo imprima correctamente en su propia terminal.

Bloque 4: Caso real:

Caso: Un usuario (Emisor) envía un mensaje a un grupo donde hay otros 2 usuarios (Receptores). El Servidor actúa como intermediario: recibe el mensaje, busca quiénes están en el grupo y reenvía el mensaje a cada uno de forma independiente.

Diagrama de Secuencia:



Instrucciones:

1. **Programar el Servidor:** Debe mantener una lista de `Sockets` activos de los receptores.
2. **Implementar la Difusión (Broadcast):** Al recibir un mensaje del Emisor, el servidor debe iterar sobre la lista y enviar el texto a cada Receptor.
3. **Manejo de Fallos:** Si un Receptor se desconectó (socket cerrado), el servidor debe eliminarlo de la lista y continuar enviando al resto sin fallar.

Código base para el alumno:

```
// Servidor de Mensajería de Grupo

public class WhatsappServer {

    // Lista para gestionar a los miembros del grupo

    private static List<PrintWriter> clientes = new ArrayList<>();

    public static void main(String[] args) throws IOException {

        ServerSocket servidor = new ServerSocket(5000);

        System.out.println("Servidor de grupo iniciado...");

        while (true) {

            Socket socket = servidor.accept();

            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
```

```
            clientes.add(out); // Agregar nuevo miembro al grupo

            // Tarea: Crear un hilo para escuchar los mensajes de este cliente

            // y reenviarlos a TODOS los demás miembros de la lista 'clientes'.

        }

    }

}
```